OCR API Deployment Guide (Ubuntu)

FastAPI + Celery + PostgreSQL + Redis behind Nginx → http://192.168.6.128/docs

Setup: Python OCR API (`app.main:app`, torch, PaddleOCR, EasyOCR, QARI/Qwen2-VL, Kerberos SSO) developed on Windows and deployed directly on Ubuntu 22.04 / 24.04 (no Docker). Project folder `/opt/ocr_api/ocr_api`, virtual environment `mvenv`, files copied with FileZilla. Replace `youruser` with your Ubuntu login name.

What changes from RHEL / Rocky: `apt` instead of `dnf` (no EPEL/CRB); Python 3.11 comes from the deadsnakes PPA; there is no SELinux, so every `setsebool` / `semanage` / `restorecon` step is dropped; the firewall is `ufw`; Nginx sites go in `sites-available` / `sites-enabled` and you disable the default site by removing a link instead of editing `nginx.conf`; PostgreSQL initialises itself on install; the Redis service is `redis-server` and the SSH service is `ssh`. Ubuntu blocks root login by default, so you log in as your own user and use `sudo`.

Step 0: Size the VM

The install and the app both need a lot of memory and disk. Too little RAM kills pip ("Out of memory: Killed process"); too little disk gives "No space left on device".

Item	Approximate size
venv (torch, paddle, transformers, OpenCV…)	5–7 GB
QARI / Qwen2-VL-2B weights	~5 GB
EasyOCR + Paddle models	0.5–1 GB
OS, PostgreSQL, logs, uploads	several GB

Give the VM **at least 8 GB RAM (16 GB is better)** and **at least 40 GB disk** (VMware → VM Settings). When installing Ubuntu, let the root filesystem `/` use the whole disk. Check after boot:

```
free -h
df -h
```

Step 1: Install system packages

```
sudo apt update
sudo apt install -y \
  build-essential pkg-config \
  libkrb5-dev krb5-user \
  tesseract-ocr tesseract-ocr-eng tesseract-ocr-ara \
  libgl1 libglib2.0-0 libgomp1 \
  libsm6 libxext6 libxrender1 \
  postgresql redis-server nginx \
  unzip curl chrony openssh-server
```

Package	Needed by
`build-essential`, `libkrb5-dev`	`pyspnego[kerberos]`, which compiles `gssapi` on Linux (Windows used SSPI). Missing `libkrb5-dev` gives the "krb5-config not found" pip error.
`krb5-user`	`/etc/krb5.conf` and the `kinit` / `klist` tools
`tesseract-ocr` + language packs	`pytesseract` is only a wrapper; the binary is `/usr/bin/tesseract`. Add other `tesseract-ocr-xxx` packs if needed.
`libgl1`, `libglib2.0-0`, `libsm6` …	OpenCV. PaddleOCR pulls in the non-headless `opencv-contrib-python`, which needs `libGL`.
`libgomp1`	OpenMP runtime for torch and paddlepaddle

Package	Needed by
`chrony`	Time sync; Kerberos fails if the clock drifts from the domain
`openssh-server`	SSH / SFTP access for FileZilla

Make sure SSH is running (FileZilla needs it):

```
sudo systemctl enable --now ssh
```

Step 2: Install Python 3.11

The requirements are tested on Python 3.11. Ubuntu 22.04 ships 3.10 and 24.04 ships 3.12, so install 3.11 from the deadsnakes PPA:

```
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev
python3.11 --version
```

Don't change the system default `python3` (e.g. with `update-alternatives`). Ubuntu's own tools rely on it. Always create the venv with `python3.11` explicitly; see which versions exist with `ls /usr/bin/python3*`.

Step 3: Transfer the project from Windows

1. Zip the project without the Windows venv (a Windows venv does not work on Linux). In PowerShell on Windows:

```
cd C:\path\to\ocr_api
tar -a -c -f ..\ocr_api.zip `
  --exclude=.venv --exclude=venv --exclude=mvenv `
  --exclude=__pycache__ --exclude=.git *
```

2. Upload with FileZilla (Quickconnect bar):

- Host: `sftp://192.168.6.128` (the `sftp://` prefix matters; plain FTP is not running)
- Username / Password: your Ubuntu user (not root), Port: `22`
- Drag `ocr_api.zip` to your home folder `/home/youruser/`. Choose **Overwrite** if asked.

Command-line alternative (PowerShell): `scp ..\ocr_api.zip youruser@192.168.6.128:~/`

3. Unzip into /opt on the server and give your user ownership:

```
sudo mkdir -p /opt/ocr_api
sudo chown youruser:youruser /opt/ocr_api
unzip -o ~/ocr_api.zip -d /opt/ocr_api/ocr_api
ls /opt/ocr_api/ocr_api        # app/, base.txt, cpu.txt ... should be here
```

`-o` overwrites without asking; `unzip -l file.zip` lists contents without extracting.

Never put the project under `/var/www/html` (Nginx's public web root on Ubuntu). Anyone could download the source, `.env` secrets and model files from a browser. Nginx only proxies to uvicorn, so the code belongs in `/opt`.

4. Copy model files. Copy your QARI / Qwen2-VL weights folder the same way. The app runs offline (automatic downloads disabled), so also copy `C:\Users\<you>\.EasyOCR` and `C:\Users\<you>\.paddlex` to `/home/youruser/` (the home of the user that runs the service).

Step 4: Create the venv and install packages

```
cd /opt/ocr_api/ocr_api
python3.11 -m venv mvenv
source mvenv/bin/activate
python --version              # must say 3.11.x
pip install --upgrade pip wheel
pip install --no-cache-dir -r base.txt -r cpu.txt     # or -r gpu.txt
```

`activate` must be **sourced** (`source mvenv/bin/activate` or `. mvenv/bin/activate`). Running it directly gives "Permission denied", and `sudo` cannot run it. Leave with `deactivate`. A venv is tied to the Python that created it and cannot be moved: if it was made with the wrong Python or in the wrong folder, `rm -rf mvenv` and create it again.

If pip only finds old versions (e.g. peft 0.17.1, pip 21.x), the venv was built with the wrong Python. Recreate it with `python3.11`.

If pip is killed (out of memory): add swap, then install torch first. Ubuntu may already have `/swap.img`; check with `swapon --show`.

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab     # keep after reboot

pip install --no-cache-dir torch==2.5.1 torchvision==0.20.1 \
  --index-url https://download.pytorch.org/whl/cpu
pip install --no-cache-dir -r base.txt -r cpu.txt
```

If the disk is full ("No space left on device"):

```
df -h                         # check Use% for /
pip cache purge
rm -rf ~/.cache/pip /tmp/pip-*
sudo apt clean
sudo apt autoremove -y
```

A swap file also uses disk; shrink it to 2 GB if needed (`sudo swapoff /swapfile`, `sudo rm /swapfile`, then recreate with `-l 2G`). To grow the disk, expand it in VMware, then send `df -h` and `lsblk` output to get the exact resize commands for your layout.

Step 5: Verify the install

```
python - <<'PY'
import torch, cv2, paddle, easyocr, transformers, gssapi, pytesseract
print('torch', torch.__version__)
print('tesseract', pytesseract.get_tesseract_version())
PY
```

If an `ImportError: libXXX.so: cannot open shared object file` appears, find the package that provides it:

```
sudo apt install -y apt-file && sudo apt-file update
apt-file search libNAME.so
```

Step 6: PostgreSQL and Redis

On Ubuntu, PostgreSQL creates and initialises its cluster during install, so there is no `postgresql-setup --initdb`.

```
sudo systemctl enable --now postgresql redis-server

sudo -u postgres createuser --pwprompt ocr_user
```

```
sudo -u postgres createdb -O ocr_user ocr_db
redis-cli ping                # should answer PONG
```

Use your own database/user names, then update the connection settings in `.env` (e.g. `postgresql://ocr_user:PASSWORD@127.0.0.1:5432/ocr_db`, `redis://127.0.0.1:6379/0`).

Step 7: Fix Windows-specific settings

- **Tesseract path:** if code sets `pytesseract.tesseract_cmd = r"C:\Program Files\..."`, make it configurable; on Ubuntu it is `/usr/bin/tesseract`.
- **Paths and case:** Linux is case-sensitive; backslash paths in config or code fail. Update model, upload and log folders.
- **Environment:** set the app to production in `.env` before real use (the startup log shows `env=development`).
- **Kerberos SSO:** create a keytab for `HTTP/your-server.domain@REALM` on AD, copy it to the server, set `KRB5_KTNAME`, configure `/etc/krb5.conf` with your realm and KDC. DNS and clock must match the domain (chrony).
- **Celery:** `--pool=solo` from Windows is not needed; the default prefork pool works. Keep concurrency low, as each process loads its own OCR models.
- **GPU + Paddle:** `gpu.txt` installs only CUDA torch; PaddleOCR stays on CPU unless you add `paddlepaddle-gpu` from Paddle's own index.

Step 8: Test run by hand

The Windows command works the same on Linux, from the folder that contains `app/` with the venv active:

```
cd /opt/ocr_api/ocr_api
source mvenv/bin/activate
uvicorn app.main:app --reload --port 8000                # local only (127.0.0.1)
```

To test from your Windows browser, listen on all interfaces and open the port temporarily:

```
sudo ufw allow 8000/tcp        # only needed if ufw is active
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Open `http://192.168.6.128:8000/docs`. Stop with Ctrl+C when done, and close the port again with `sudo ufw delete allow 8000/tcp`. `--reload` is for development only.

Step 9: Run the API as a systemd service

```
sudo nano /etc/systemd/system/ocr-api.service
```

Paste, then save with **Ctrl+O**, **Enter**, **Ctrl+X**:

```
[Unit]
Description=OCR API (uvicorn)
After=network.target postgresql.service redis-server.service

[Service]
User=youruser
Group=youruser
WorkingDirectory=/opt/ocr_api/ocr_api
ExecStart=/opt/ocr_api/ocr_api/mvenv/bin/uvicorn app.main:app \
    --host 127.0.0.1 --port 8000 --workers 1
Restart=always

[Install]
WantedBy=multi-user.target
```

Keep `--workers 1` for now: each worker loads its own copy of the OCR models into RAM. The venv binaries are called directly, so no activation is needed in the service.

```
sudo systemctl daemon-reload
sudo systemctl enable --now ocr-api
systemctl status ocr-api
curl -I http://127.0.0.1:8000/docs        # should return 200 OK
journalctl -u ocr-api -f                  # live logs
```

Celery worker uses a second service in the same way. Template (replace `app.worker` with your Celery app module, as used in your Windows command):

```
# /etc/systemd/system/ocr-worker.service
[Unit]
Description=OCR Celery worker
After=network.target postgresql.service redis-server.service

[Service]
User=youruser
Group=youruser
WorkingDirectory=/opt/ocr_api/ocr_api
ExecStart=/opt/ocr_api/ocr_api/mvenv/bin/celery -A app.worker worker \
    --loglevel=info --concurrency=1
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload && sudo systemctl enable --now ocr-worker
```

Step 10: Nginx reverse proxy

```
sudo nano /etc/nginx/sites-available/ocr_api

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name _;

    client_max_body_size 50M;          # allow large PDF/image uploads

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 300s;       # OCR can be slow
    }
}
```

Enable it and disable Ubuntu's default site (this replaces commenting out the `server { }` block in `nginx.conf` on Rocky; leave `nginx.conf` untouched on Ubuntu):

```
sudo ln -s /etc/nginx/sites-available/ocr_api /etc/nginx/sites-enabled/ocr_api
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t                  # must say "syntax is ok" and "test is successful"
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

No SELinux on Ubuntu, so the Rocky step `setsebool -P httpd_can_network_connect 1` is not needed; Nginx can reach uvicorn on 127.0.0.1:8000 out of the box.

Step 11: Firewall (ufw)

```
sudo ufw status
```

If **inactive**, ports are already open. If **active**:

```
sudo ufw allow 'Nginx HTTP'          # or 'Nginx Full' for HTTP + HTTPS
sudo ufw reload
```

> **Before** turning ufw on (`sudo ufw enable`), run `sudo ufw allow OpenSSH`, or you will lock yourself out of SSH and FileZilla.

Step 12: Test

Open `http://192.168.6.128/docs` from Windows with no port number. Press Ctrl+F5 to bypass a cached error page.

Troubleshooting

Symptom	Fix
"Welcome to nginx" or nginx 404 page	The default site is answering. Check `ls -l /etc/nginx/sites-enabled/` (only `ocr_api` should be there) and `sudo nginx -T 2>/dev/null | grep -nE "server_name|listen|proxy_pass"`; reload Nginx.
502 Bad Gateway	uvicorn is not running: `systemctl status ocr-api`, `curl -I http://127.0.0.1:8000/docs`, `journalctl -u ocr-api -n 50`.
Page does not load at all	`sudo ufw status` (Step 11); `systemctl status nginx`.
FileZilla connection refused	`sudo systemctl enable --now ssh`; `sudo ufw allow OpenSSH`; use `sftp://` and your normal user, not root.
pip: krb5-config not found / gssapi build fails	`sudo apt install -y libkrb5-dev krb5-user build-essential python3.11-dev`, check `krb5-config --version`, rerun pip.
pip finds only old versions	venv made with the wrong Python. Recreate with `python3.11 -m venv mvenv`.
pip killed / Out of memory	Add swap (Step 4) and give the VM more RAM.
No space left on device	Clear caches (Step 4) and expand the disk.
ImportError: libGL.so.1	`sudo apt install -y libgl1 libglib2.0-0 libgomp1`.
413 Request Entity Too Large	Raise `client_max_body_size` in the Nginx site, reload.
504 Gateway Timeout on large files	Raise `proxy_read_timeout`.

Logs:

```
journalctl -u ocr-api -f
journalctl -u ocr-worker -f
sudo tail -f /var/log/nginx/error.log
```

RHEL / Rocky → Ubuntu command map

RHEL / Rocky	Ubuntu
`dnf install -y epel-release; dnf config-manager --set-enabled crb`	`not needed (sudo add-apt-repository ppa:deadsnakes/ppa for Python 3.11)`
`dnf install -y python3.11 python3.11-devel`	`sudo apt install -y python3.11 python3.11-venv python3.11-dev`
`gcc gcc-c++ make`	`build-essential`
`krb5-devel krb5-workstation`	`libkrb5-dev krb5-user`
`tesseract tesseract-langpack-eng tesseract-langpack-ara`	`tesseract-ocr tesseract-ocr-eng tesseract-ocr-ara`
`mesa-libGL glib2 libgomp`	`libgl1 libglib2.0-0 libgomp1`
`postgresql-server + postgresql-setup --initdb`	`postgresql (initialised on install)`
`redis / systemctl enable --now redis`	`redis-server / sudo systemctl enable --now redis-server`
`systemctl enable --now sshd`	`sudo systemctl enable --now ssh`
`dnf install -y unzip`	`sudo apt install -y unzip`
`dnf clean all`	`sudo apt clean`
`dnf provides '*/libNAME.so*'`	`apt-file search libNAME.so`
`setsebool -P httpd_can_network_connect 1`	`not needed (no SELinux)`
`semanage fcontext ... ; restorecon -Rv ...`	`not needed (no SELinux)`
`firewall-cmd --permanent --add-service=http; --reload`	`sudo ufw allow 'Nginx HTTP'`
`firewall-cmd --add-port=8000/tcp`	`sudo ufw allow 8000/tcp`
`firewall-cmd --permanent --add-service=ssh`	`sudo ufw allow OpenSSH`
`/etc/nginx/conf.d/ocr_api.conf`	`/etc/nginx/sites-available/ocr_api + link in sites-enabled`
`comment out server { } in /etc/nginx/nginx.conf`	`sudo rm /etc/nginx/sites-enabled/default`
`/usr/share/nginx/html (web root)`	`/var/www/html (web root)`
`root login over SSH/SFTP`	`your own user + sudo`
`echo '...' >> /etc/fstab (as root)`	`echo '...' | sudo tee -a /etc/fstab`

Appendix: Docker or GPU instead

Docker: if the project's Dockerfile installs the system libraries, the host needs only Docker Engine + Compose:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
```

```
CODENAME=$(. /etc/os-release && echo $VERSION_CODENAME)
ARCH=$(dpkg --print-architecture)
echo "deb [arch=$ARCH signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu $CODENAME stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER            # log out and back in
```

GPU profile: install the NVIDIA driver with `sudo ubuntu-drivers install`, reboot, check `nvidia-smi`, then install with `-r base.txt -r gpu.txt`. The CUDA toolkit is not needed on the host (the cu121 torch wheels bundle the runtime). With Docker, also install the NVIDIA Container Toolkit.